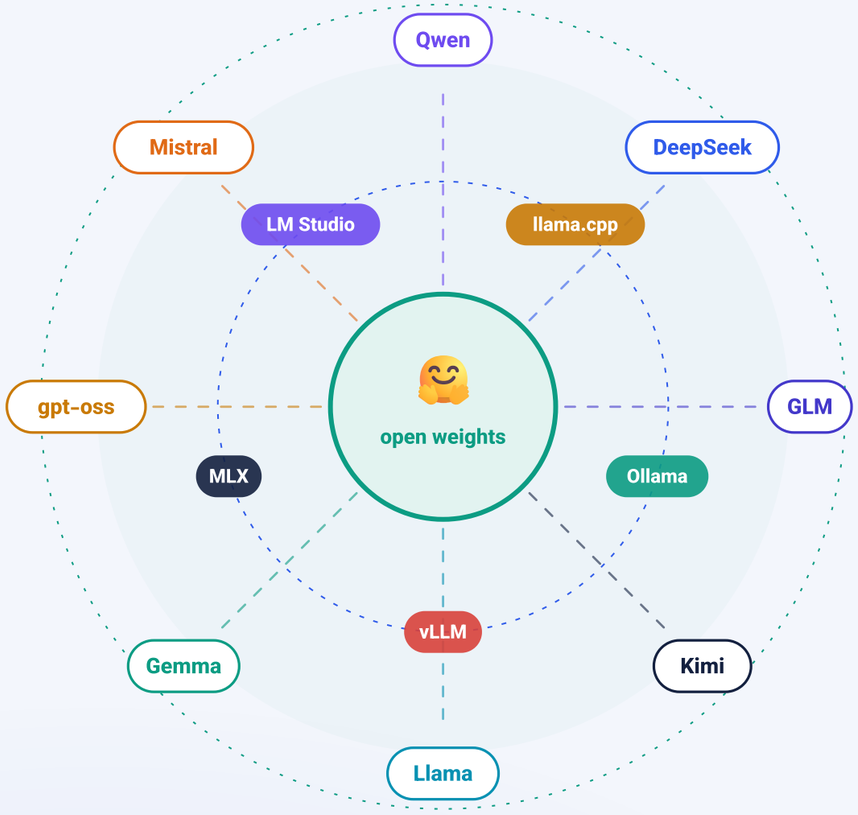


SESSION 05 // OPEN MODELS · LOCAL LLMs

جلسه ۵

مدل‌های متن‌باز و LLM محلی

از چشم‌انداز مدل‌های باز و مجوزها تا GGUF، Quantization، پنج راه اجرای محلی (llama.cpp، Ollama، LM Studio، vLLM، MLX)، سخت‌افزار، GPUها و استقرار امن؛ و پروژهٔ امروز: یک Workflow مالی چند-مدلی با LangGraph.

رضا شکرزاد · مدرس هوش مصنوعی و علم داده بوتکمپ ایجنت‌های هوش مصنوعی · کافه‌تدریس

AGENDA

فهرست امروز

تا امروز مدل‌ها را از طریق API صدا زدیم؛ امروز مدل را **به خانه می‌آوریم**: لپ‌تاپ، سرور سازمان یا GPU اجاره‌ای.

01
پروژهٔ امروز
Workflow مالی چند-مدلی، SPEC و پرامپت

02
متن‌باز یعنی چه؟
Open Source، Open Weight، مجوزها

03
چشم‌انداز مدل‌ها
بسته‌شدن فاصله باز و بسته

04
چرا LLM محلی؟
حریم خصوصی، هزینه، آفلاین

05
رتبه‌بندی و جامعه‌ها
Leaderboardها و Communityها

06
نام‌گذاری و انواع مدل
Base، Instruct، A3B، Q4_K_M

07
Dense و MoE و حافظه
چقدر VRAM لازم است؟

08
GGUF و Quantization
فرمت فایل و کوانتیزه‌سازی

09
پنج راه اجرای محلی
کدام برای چه کاری؟

10
llama.cpp، Ollama، LM Studio
از موتور تا رابط گرافیکی

11
vLLM و SGLang
Production و PagedAttention

12
MLX-LM و Apple Silicon
Unified Memory

13
سخت‌افزار: پهنای باند
سرعت = پهنای باند ÷ حجم

14
Ryzen AI Max
حافظه یکپارچه ۱۲۸ گیگ

15
Mini PCها و کلاستر
برندها و اتصال چند دستگاه

16
تراشه‌ها و GPUهای NVIDIA
GPU، ASIC، FPGA، NPU

17
استقرار، Fine-tune، امنیت
از لپ‌تاپ تا سرور

18
RLHF، Harness و پروژه‌ها
...JEV، Hermes، Whisper، OpenHands

19
مدل محلی در پروژهٔ مالی
پروژهٔ مالی و نصب OpenCode

20
چک‌لیست‌های دوره
مدل‌سازی، معماری، انتخاب مدل، امنیت، قانون

WHAT WE BUILD TODAY

امروز چه می‌سازیم؟ یک Workflow مالی با چهار نقش مدل

یک گراف کوچک با LangGraph: سند را می‌گیرد، طبقه‌بندی می‌کند، با مدل محلی استخراج و با دو مدل API تحلیل و بررسی می‌کند. هدف آموزش Orchestration و مقایسهٔ نقش مدل‌هاست، نه یک سیستم مالی واقعی.



همه چیز روی یک State مشترک

هر گره در LangGraph روی همان State می‌نویسد: سند، دسته، فیلدها، تحلیل، اعتبارسنجی، خطاها، لاگ، تأخیر و توکن. دسته‌بندی JEV مسیر را تعیین می‌کند (Routing شرطی).

قواعد این دمو

- هیچ فراخوانی مدل جعلی یا موفقیت ساختگی نیست؛ خطا واضح نمایش داده می‌شود
- مدل محلی هرگز بی‌صدا با مدل دیگری جایگزین نمی‌شود
- اعتبارسنجی مدل فقط مشورتی است؛ فقط سند فرضی و آموزشی

فایل SPEC.md

مشخصات کامل پروژه (۱۴ بخش). همراه پرامپت اسلاید بعد به Code Agent بدهید.

دانلود SPEC.md

کپی متن

PROMPT · MULTI-AGENT FINANCE DEMO

پرامپت کامل برای Code Agent: دموی مالی چند-Agent

کپی پرامپت | دانلود PROMPT.md

finance-agent-demo · prompt

Build a Minimal Multi-Agent Financial Workflow Demo

OBJECTIVE

Build the simplest working multi-agent orchestration workflow for a corporate finance teaching demo. The purpose is to demonstrate orchestration logic, model interaction, conditional routing, and model comparison—not to build a production finance system.

Prioritize functionality, minimal code, and easy local execution.

1. TECHNOLOGY STACK

- Python
- LangGraph for orchestration
- Streamlit for the user interface and dashboard
- Ollama for running a small local CPU model
- Two configurable LLM API providers
- A classifier model called JEV, accessed through a configurable API
- Local files for input documents
- No database, authentication, Docker, or cloud deployment

Use the simplest compatible libraries and avoid unnecessary abstractions.

2. WORKFLOW

Implement this pipeline:

STEP 1 — LOAD DOCUMENTS

- Read PDF, CSV, and TXT files from a local `documents/` folder.
- Also provide a simple file uploader in Streamlit.
- Extract text or tabular content using lightweight Python libraries.

STEP 2 — CLASSIFY AND ROUTE

- Send document text to JEV.
- Classify it into one of these categories:
  - Invoice
  - Expense
  - Financial report
  - Other
- Route the document to the appropriate processing path.
- If classification fails, use a fallback rule-based classifier.

STEP 3 — EXTRACT FINANCIAL INFORMATION

- Use a small, CPU-friendly Ollama model, configurable through environment variables.
- Extract relevant fields such as document type, date, vendor, amount, currency, tax, and description.
- Return structured JSON.
- If the local model is unavailable, show a clear error and allow the user to retry. Do not silently substitute another model.

STEP 4 — ANALYZE FINANCES

- Send the extracted information to API Model 1.
- Ask it to identify financial insights, anomalies, missing information, and potential risks.
- Require a structured response.

STEP 5 — VALIDATE RESULTS

- Send the original extracted data and analysis to API Model 2.
- Ask it to check arithmetic, consistency, unsupported claims, and missing fields.
- Return a validation status and brief explanation.
- Treat model validation as an advisory check, not proof of financial correctness.

STEP 6 — FINAL OUTPUT

- Combine classification, extraction, analysis, and validation into a final result.
- Display the complete output in the dashboard.
- Keep all intermediate results accessible for teaching.

3. ORCHESTRATION REQUIREMENTS

Use LangGraph with a shared state containing:

- Document name and extracted text
- Classification result
- Extracted financial data
- Analysis result
- Validation result
- Errors
- Execution logs
- Model latency and token usage, when available

Implement:

- Sequential execution where appropriate
- Conditional routing based on document classification
- A shared state passed between nodes
- Error handling for failed model calls
- A visible record of the execution order

Keep the graph small and readable. Do not introduce additional agents or complex planning loops.

4. SIMPLE STREAMLIT INTERFACE

Create one Streamlit page with three sections.

A. INPUT

- Upload a document or select a file from `documents/`.
- A button labelled "Run Workflow".
- Show the selected document name.

B. EXECUTION

Display the execution steps as a simple status list:

1. Classification
2. Document extraction
3. Financial analysis
4. Validation
5. Complete

Show each step's status and basic execution logs. Display errors clearly.

OPEN SOURCE VS OPEN WEIGHT

«متن‌باز» در دنیای LLM دقیقاً یعنی چه؟

بیشتر مدل‌های «Open Source» در واقع **Open Weight** هستند: وزن‌ها قابل دانلود است، اما داده و کد آموزش نه.

🔒 Closed · بسته

فقط از طریق API؛ وزن‌ها در دسترس نیست

GPT، Claude، Gemini

⚖️ Open Weight · وزن باز

وزن‌ها قابل دانلود و اجرا؛ داده و کد آموزش منتشر نشده

Llama، Qwen، DeepSeek، GLM، Kimi، Gemma، gpt-oss، Mistral

🌳 Fully Open · کاملاً باز

وزن + داده آموزش + کد + جزئیات؛ قابل بازتولید

OLMo (AI2)، Pythia، SmolLM

مجوز	یعنی چه؟	نمونه
Apache 2.0 / MIT	آزاد برای استفاده تجاری، تغییر و توزیع	Qwen، gpt-oss، DeepSeek، GLM
Modified MIT	MIT با شرط کوچک (مثلاً ذکر نام در مقیاس بزرگ)	Kimi
Community License	استفاده تجاری با محدودیت (مثلاً سقف کاربر، نام‌گذاری)	Llama
Custom Terms	شرایط اختصاصی سازنده؛ باید خواند	Gemma، برخی مدل‌های Mistral
Research / Non-commercial	فقط پژوهش؛ استفاده تجاری ممنوع	برخی مدل‌های تخصصی

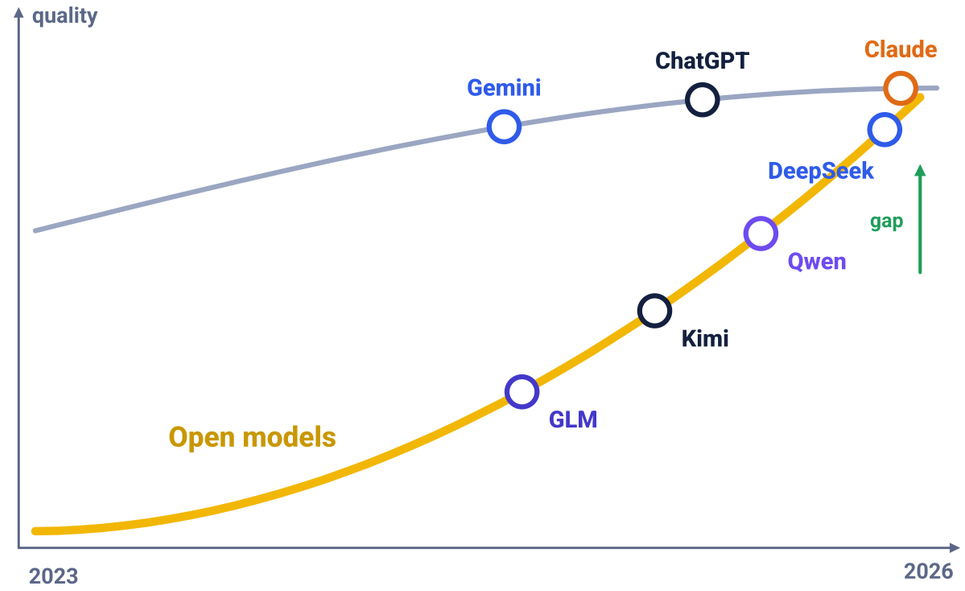
⚠️ مجوز را برای **هر نسخه** جداگانه بررسی کنید؛ یک خانواده مدل ممکن است در نسخه‌های مختلف مجوز متفاوت داشته باشد. Fine-tuneهای جامعه هم مجوز مدل پایه را به ارث می‌برند.

Agentic برای پروژه‌های سازمانی (مثل دموی مالی امروز) مدلی با **Apache 2.0** یا **MIT** انتخاب کنید تا استفاده تجاری، Fine-tune و استقرار On-premise بدون ابهام حقوقی باشد.

LLM LANDSCAPE

چشم‌انداز: فاصله مدل‌های باز و بسته در حال بسته شدن است

از ۲۰۲۳ تا ۲۰۲۶ مدل‌های باز (به‌ویژه از آزمایشگاه‌های چینی) با سرعت بیشتری از مدل‌های بسته پیشرفت کرده‌اند و امروز در بسیاری از وظایف نزدیک به مرز دانش هستند.



کیفیت مدل‌های باز (زرد) در برابر مدل‌های بسته (خط‌چین)، ۲۰۲۳ تا ۲۰۲۶

DeepSeek — DeepSeek
V (MoE بزرگ)، R (استدلال)، Distillها
MIT

Qwen — Alibaba
۰.۶B تا MoE بسیار بزرگ؛ Coder، VL، Thinking
Apache 2.0

Kimi — Moonshot AI
MoE تریلیون پارامتری؛ Agentic و Tool Use
Modified MIT

GLM — Zhipu / Z.ai
Agentic و کدنویسی قوی؛ مدل باز پیشرو در جدول‌ها
MIT

Gemma — Google
مدل‌های کوچک و کارآمد برای لپ‌تاپ و موبایل
Gemma Terms

Llama — Meta
8B تا مدل‌های بزرگ؛ اکوسیستم بسیار گسترده
Llama License

Mistral — Mistral AI (فرانسه)
Small، Medium، Codestral؛ اروپایی
Apache / Custom

gpt-oss — OpenAI
مدل‌های باز استدلالی MoE
Apache 2.0

OLMo — AI2
کاملاً باز: داده، کد، Checkpoint
Apache 2.0

Phi — Microsoft
مدل‌های کوچک با داده با کیفیت
MIT

WHY LOCAL LLMs?

چرا LLM را محلی اجرا کنیم؟

۱ ۱۰۰٪ حریم خصوصی داده

داده هرگز از دستگاه یا شبکه سازمان خارج نمی‌شود: سند مالی، پرونده پرسنلی، قرارداد، داده مشتری

۲ بدون هزینه اشتراک

هزینه Token صفر؛ فقط سخت‌افزار و برق. برای حجم بالا و ثابت، ارزان‌تر

۳ کارکرد آفلاین

بدون اینترنت، بدون تحریم و قطعی سرویس؛ در کارخانه، کشتی، منطقه دورافتاده

✅

مزیت‌های دیگر

- **کنترل نسخه**: مدل بی‌خبر عوض یا حذف نمی‌شود
- **Latency** قابل پیش‌بینی، بدون Rate Limit
- **سفارشی‌سازی**: Fine-tune روی داده خودتان
- **انطباق**: نگهداری داده در کشور یا سازمان (Data Residency)
- یادگیری عمیق‌تر از نحوه کار مدل‌ها

⚠️

هزینه‌های پنهان

- **سخت‌افزار**: GPU یا حافظه زیاد گران است
- **کیفیت**: مدل‌های کوچک محلی هنوز از بهترین مدل‌های بسته عقب‌ترند
- **عملیات**: به‌روزرسانی، پایش، امنیت با خودتان
- **مقیاس**: کاربران هم‌زمان زیاد = سرور جدی
- برق، خنک‌سازی، نویز

API PRICING · VS · IN-HOUSE INFRA

قیمت API مدل‌ها در برابر هزینهٔ اجرای درون‌سازمانی

دلار به‌ازای ۱ میلیون توکن؛ ستون آخر: هزینهٔ ۱۰۰۰ اجرای یک Workflow نمونه (2K ورودی + 0.5K خروجی)، مرتب از ارزان به گران.

Provider	Model	Context	Input / 1M	Output / 1M	Cached Input	اجرا ۱۰۰۰
Z.ai	**glm-5.3-flash**	1.0M	$0	$0	$0.0087	**$0**
DeepSeek	**deepseek-chat (V4)**	164K	$0.014	$0.028	—	**$0.04**
Mistral AI	**mistral-nemo**	131K	$0.017	$0.022	$0.0027	**$0.04**
OpenAI	**gpt-oss-20b**	131K	$0.018	$0.09	$0.003	**$0.08**
Google	**gemini-2.5-flash-lite**	1.0M	$0.05	$0.2	$0.005	**$0.2**
OpenAI	**gpt-5-nano**	400K	$0.05	$0.4	$0.005	**$0.3**
DeepSeek	**DeepSeek V4 Flash**	1.0M	$0.14	$0.28	$0.0028	**$0.42**
Anthropic	**Claude Haiku 4.5**	200K	$1	$5	$0.1	$4.5
Google	**Gemini 2.5 Flash**	1.0M	$3	$5	—	$8.5
OpenAI	**o4-mini**	128K	$11	$8.8	—	**$26.4**
xAI	**Grok 4.20**	128K	$20	$12	—	**$46**
Anthropic	**Claude Sonnet 5**	200K	$20	$20	—	**$50**

Workflow نمونه	مدل باز	زیرساخت	هزینهٔ تقریبی (برآورد)
دستیار داخلی و RAG ۵۰ کاربر	مدل ۸ تا ۱۴B (Q4)	یک RTX 4090/5090 یا Mac mini M4 Pro	**خرید حدود ۲ تا ۳٫۵ هزار دلار · برق ماهانه ۳۰ تا ۶۰ دلار**
استخراج اسناد (دموی مالی) ۱۰ هزار سند در روز ≈ ۳۰M توکن	MoE مثل 30B-A3B یا gpt-oss-20b	یک GPU ۳۲GB با vLLM (فقط ≈ ۳۵۰ tok/s لازم است)	**خرید حدود ۴ تا ۵ هزار دلار · یا اجارهٔ ماهانه ۴۰۰ تا ۶۰۰ دلار**
Agent برای تیم ۲۰ کاربر هم‌زمان	MoE ۱۲۰B یا Dense ۷۰B	۱ تا ۲ کارت ۹۶GB یا ۲× H100	**خرید حدود ۱۰ تا ۲۵ هزار دلار · اجارهٔ ماهانه ۱٫۵ تا ۴٫۵ هزار دلار**
مدل مرزی باز GLM، Kimi، DeepSeek بزرگ	MoE چند صد میلیارد	گره ۸×H200/B200 (یا چند Mac Studio برای ۱ تا ۳ کاربر)	**خرید حدود ۲۵۰ تا ۴۰۰ هزار دلار · اجارهٔ ماهانه ۱۵ تا ۲۵ هزار دلار**

نتیجه (۳۰M توکن در روز): gpt-oss-20b حدود ۲۷ دلار در ماه، از هر سرور خودتان ارزان‌تر است؛ Haiku حدود ۱٬۵۰۰ و Sonnet حدود ۱۸٬۰۰۰ دلار در ماه می‌شود و GPU زود خودش را جبران می‌کند. دلیل اصلی اجرای درون‌سازمانی معمولاً حریم خصوصی و کنترل است، نه ارزانی. اعداد برآورد تقریبی‌اند.

RANKING & COMMUNITIES

رتبه‌بندی مدل‌ها و جامعه‌های LLM محلی

برای انتخاب مدل، هم جدول‌های رتبه‌بندی را ببینید و هم تجربه واقعی جامعه را؛ و در نهایت روی داده خودتان Eval بگیرید.

Leaderboardها

- **Agent Arena**: رتبه‌بندی پویای مدل‌ها در انجام وظایف Agentic واقعی (قابلیت اطمینان ابزار، تکمیل وظیفه، Steerability) arena.ai/leaderboard/agent
- **BenchLM**: مقایسه مدل‌ها با مجوز (Open/Closed)، قیمت، Context، سرعت benchlm.ai
- **Artificial Analysis**: کیفیت، سرعت و قیمت ارائه‌دهنده‌ها artificialanalysis.ai
- **SWE-bench**، **LiveBench**: بنچمارک‌های به‌روز و کدنویسی

در جدول‌های ژوئیه ۲۰۲۶، **GLM-5.2** (Z.ai، مجوز MIT، قابل Self-host) بالاترین مدل باز در کنار مدل‌های بسته بود. جدول‌ها هر هفته تغییر می‌کنند؛ **Leaderboard ≠ عملکرد روی داده شما**.

r/LocalLLaMA

Reddit

مرکز اصلی بحث مدل‌های قابل‌میزبانی محلی

reddit.com/r/LocalLLaMA

r/LocalLLM

Reddit

اجرای محلی LLMها

reddit.com/r/LocalLLM

x/LocalLLaMA

X

مدل‌های محلی، Self-hosting و سخت‌افزار

x.com

llama.cpp

GitHub

ggml-org/llama.cpp؛ موتور اجرای C/C++

github.com/ggml-org/llama.cpp

axolotl

GitHub

ابزار Fine-tune و جامعه پرسش و پاسخ

github.com/axolotl-ai-cloud/axolotl

Hugging Face

Hub

بزرگ‌ترین مخزن مدل‌ها و دیتاست‌ها

huggingface.co/models

Agentic

قبل از انتخاب مدل برای یک Agent، دو معیار را جدا بسنجید: کیفیت **Tool Calling** (خروجی JSON معتبر، انتخاب ابزار درست) و پایداری در **حلقه‌های طولانی**. Agent Arena دقیقاً همین را می‌سنجد.

MODEL NAMING

نام مدل را مثل یک کد بخوانید

نام فایل در Hugging Face خلاصه کامل مدل است: خانواده، نسخه، اندازه، معماری، سازنده و نوع Fine-tune.

Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive

Qwen	3.6	35B	A3B	Uncensored	HauhauCS	Aggressive
خانواده مدل	**نسخه**	**کل پارامترها**	**پارامتر فعال**	**محدودیت حذف‌شده**	**سازنده**	**سبک رفتاری**
مدل پایه ساخت Alibaba Cloud	نسخه مشخص از Qwen	B = میلیارد؛ اندازه و ظرفیت مدل	MoE: فقط ۳ میلیارد در هر Token فعال	Fine-tune برای حذف محدودیت‌های محتوایی	نام فرد یا گروهی که Fine-tune کرده	Tune شده برای پاسخ قاطع‌تر و مستقیم‌تر

پسوندهای رایج دیگر

Base	پیش‌آموزش‌دیده؛ فقط ادامه متن، برای Fine-tune
Instruct / Chat / it	تنظیم‌شده برای دستورپذیری و گفتگو
Thinking / Reasoning	قبل از پاسخ فکر می‌کند؛ کندتر و دقیق‌تر
Distill	مدل کوچکی که از خروجی مدل بزرگ یاد گرفته
Coder / VL / Embed	تخصصی: کد، بینایی-زبان، Embedding
GGUF / AWQ / MLX	فرمت و روش کوانتیزه‌سازی
Q4_K_M / 4bit	سطح کوانتیزه‌سازی

جمع‌بندی مثال: Qwen نسخه ۳.۶ با ۳۵ میلیارد پارامتر کل، معماری Mixture-of-Experts با ۳ میلیارد پارامتر فعال، بدون محدودیت محتوایی، ساخته HauhauCS و تنظیم‌شده برای لحن قاطع.

⚠️ مدل‌های **Uncensored** و Fine-tuneهای ناشناس جامعه: گاردریل ایمنی ندارند، منشأ داده آموزش نامعلوم است و برای محصول سازمانی مناسب نیستند. همیشه از مدل رسمی سازنده شروع کنید.

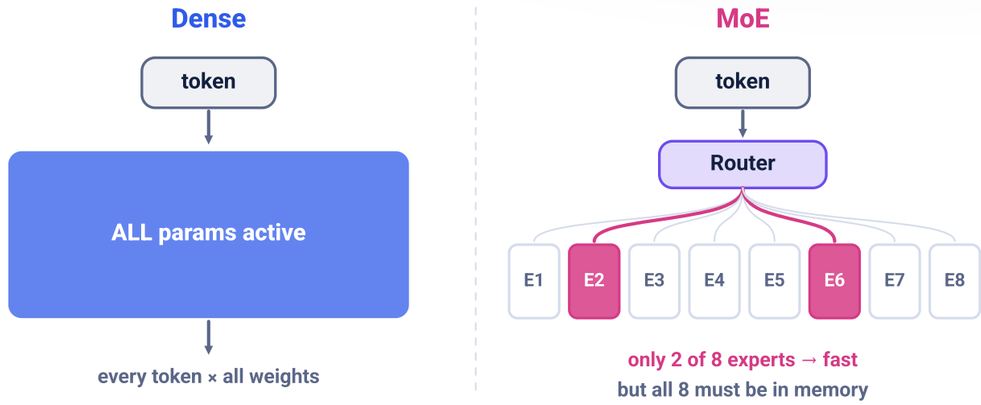
DENSE · MoE · MEMORY

Dense یا MoE؟ و چقدر حافظه لازم است؟

MoE = Mixture of Experts = ترکیب متخصص‌ها: در هر لایه چند «متخصص» وجود دارد و یک Router برای هر Token فقط چند تا را فعال می‌کند. **حافظه** به کل پارامترها بستگی دارد، **سرعت** به پارامترهای فعال.

مدل	پارامتر	FP16	Q8	Q4
Small	8B	16 GB	8 GB	4 GB
Mid	13B	26 GB	13 GB	6.5 GB
Large	30B	60 GB	30 GB	15 GB
Dense large	70B	140 GB	70 GB	35 GB
MoE 30B-A3B	30B (3B فعال)	60 GB	30 GB	15 GB
MoE 120B	120B (~5B فعال)	240 GB	120 GB	60 GB

نکته MoE: مدل 30B-A3B باید **کامل** در حافظه باشد (حدود ۱۵ گیگ در Q4)، اما در هر Token فقط حدود ۱.۵ گیگ داده خوانده می‌شود؛ پس به سرعت یک مدل ۳ میلیاردی اجرا می‌شود. به همین دلیل MoEها روی دستگاه‌های با حافظه زیاد اما پهنای باند کم عالی‌اند.



Dense: همه پارامترها برای هر Token · MoE: فقط چند متخصص

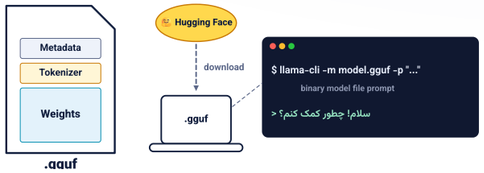
فرمول حافظه

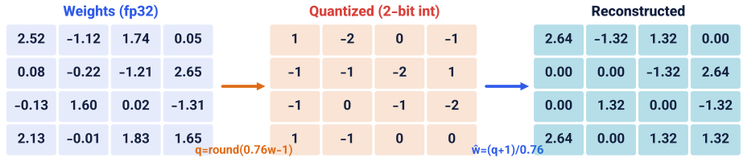
سربار ۱۰-۲۰٪ + KV Cache + حافظه ≈ پارامترها × بایت به ازای هر پارامتر

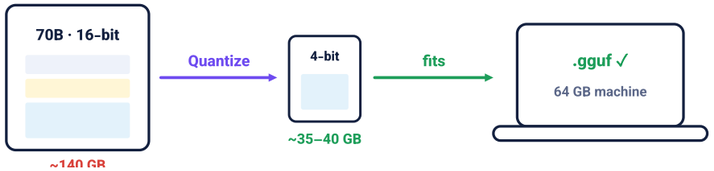
FP16 = ۲ بایت · Q8 = ۱ بایت · Q4 ≈ ۰.۵ بایت. **KV Cache** با طول Context و تعداد کاربر هم‌زمان رشد می‌کند؛ Context طولانی می‌تواند چند گیگابایت اضافه کند.

GGUF و Quantization: مدل ۷۰ میلیاردی روی لپ‌تاپ

GGUF: فرمت استاندارد یک‌فایلی برای LLMهای محلی (جانشین GGML در ۲۰۲۳). **Quantization**: ذخیره وزن‌ها با بیت کمتر (۱۶ ← ۸ ← ۴ بیت) تا حجم و حافظه چند برابر کم شود، با افت کیفیت اندک.







Quantization چطور کار می‌کند؟ (Affine)

$$q = round(s \cdot w + z) \cdot \hat{w} = (q - z) / s$$

$s = (q_{max} - q_{min}) / (w_{max} - w_{min})$ و $z = q_{min} - round(s \cdot w_{min})$. در مثال (۳۲ بیت ← ۲ بیت): $s = ۰.۷۶$ و $z = -۱$. اختلاف $w - \hat{w}$ همان **خطای کوانتیزه‌سازی** است. Jacob et al., 2017 (arXiv 1712.05877)

سطح	حجم نسبی	کیفیت	کِی؟
Q8_0	۵۰٪	تقریباً بدون افت	وقتی حافظه دارید
Q5_K_M / Q6_K	۳۵-۴۰٪	بسیار نزدیک	تعادل عالی
Q4_K_M	~۳۰٪	افت کم	پیش‌فرض رایج
Q3 / IQ2	و کمتر ۲۰٪	افت محسوس	فقط مدل‌های بزرگ

GGUF (llama.cpp) **GPTQ** **AWQ** **EXL2/EXL3** **bitsandbytes NF4** **FP8** **MLX 4-bit**

5 WAYS TO RUN LLMs LOCALLY

پنج راه اجرای LLM محلی، و کدام را انتخاب کنیم؟

بیشتر ابزارهای محلی روی یک موتور مشترک ساخته شده‌اند: **llama.cpp**. تفاوت در رابط کاربری، هدف (نمونه‌سازی یا Production) و سخت‌افزار است.

1 **llama.cpp** — موتور C/C++؛ بیشترین کنترل
CLI · Server

2 **Ollama** — یک دستور برای دانلود و اجرا
CLI · API :11434

3 **LM Studio** — اپ گرافیکی، بدون ترمینال
GUI · API :1234

4 **vLLM / SGLang** — سرور با Throughput بالا
API :8000

5 **MLX-LM** — بهینه برای Apple Silicon
CLI · Python

کدام یکی؟



همه این‌ها یک **API سازگار با OpenAI** ارائه می‌دهند: کدی که برای GPT نوشتید، با تغییر base_url روی مدل محلی اجرا می‌شود.

llama.cpp، Ollama و LM Studio: موتور و دو رابط

llama.cpp (۱۰ مارس ۲۰۲۳، Georgi Gerganov): اجرای LLM با C/C++ ساده، بدون وابستگی؛ حتی روی Raspberry Pi، لپ‌تاپ قدیمی بدون GPU و گوشی. Ollama و LM Studio روی آن ساخته شده‌اند.



1 · llama.cpp

بیشترین کنترل: انتخاب لایه‌های روی GPU، طول Context، Sampling. سرور سازگار با OpenAI روی پورت ۸۰۸۰.

```
llama-cli -m model.gguf -p "سلام"
llama-server -m model.gguf \
   --n-gpu-layers 99 --ctx-size 8192
```

github.com/ggml-org/llama.cpp

2 · Ollama

llama.cpp به‌عنوان **موتور هسته**، به‌علاوه CLI ساده، مدیریت دانلود مدل و سرور. «Docker برای LLMها». سفارشی‌سازی با **Modelfile**.

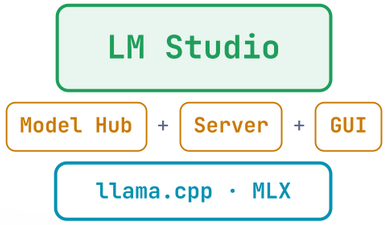
```
$ ollama run gemma4
pulling manifest...
downloading weights
server on :11434
>>>
```

```
ollama pull qwen3.6:35b-a3b
ollama list · ollama ps
curl localhost:11434/v1/chat/completions
```

ollama.com

3 · LM Studio

بدون ترمینال، بدون Config: GUI + جستجو و دانلود مدل از Hugging Face + سرور محلی، روی llama.cpp (و MLX در Mac).

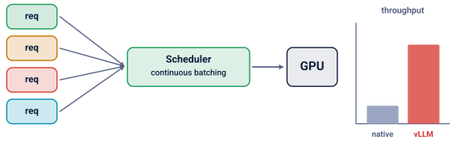


برای آزمایش سریع مدل‌ها، مقایسه کوانتیزه‌ها و نمایش به مدیران عالی است. سرور محلی روی پورت ۱۲۳۴.

lmstudio.ai

vLLM: از نمونه اولیه تا Production

Ollama برای یک کاربر روی لپ‌تاپ عالی است؛ اما وقتی ده‌ها کاربر هم‌زمان POST /v1/... می‌فرستند، به سروری نیاز دارید که GPU را کاملاً پر نگه دارد. دو نوآوری اصلی vLLM:



Continuous Batching: Scheduler درخواست‌ها را پیوسته به GPU می‌فرستد

۲ · Continuous Batching

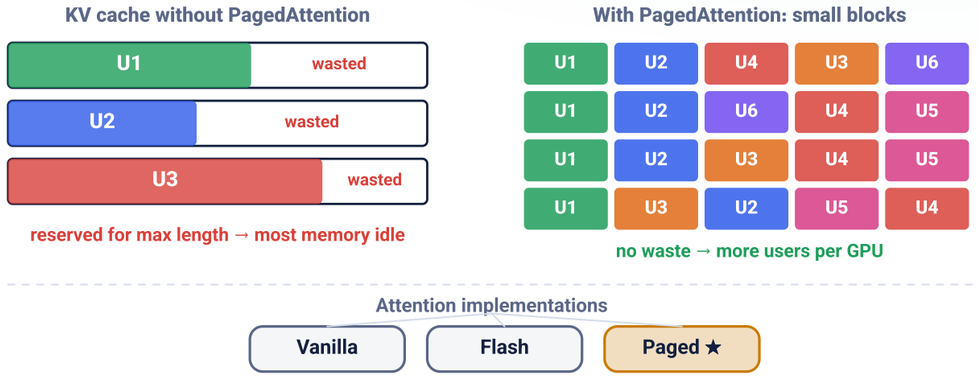
به‌جای انتظار برای تمام شدن کل Batch، به محض تمام شدن یک درخواست، درخواست بعدی جایش وارد می‌شود. نتیجه: Throughput چند برابر اجرای ساده (native).

جایگزین: SGLang

RadixAttention برای استفاده مجدد از Prefix مشترک؛ عالی برای Agentها با System Prompt ثابت. docs.sglang.io

```
pip install vllm
vllm serve Qwen/Qwen3.6-35B-A3B \
  --max-model-len 32768
```

vllm.ai · پورت پیش‌فرض ۸۰۰۰ · همچنین: TGI، TensorRT-LLM



بدون PagedAttention: حافظه رزرو و هدر می‌رود · با PagedAttention: بلوک‌های کوچک، مثل حافظه مجازی سیستم‌عامل

۱ · PagedAttention

KV Cache هر کاربر به بلوک‌های کوچک تقسیم و هر جا جای خالی بود ذخیره می‌شود (الهام از Paging در سیستم‌عامل). هدررفت حافظه تقریباً صفر ← کاربران هم‌زمان بیشتر. از خانواده **Paged**، Attention: Vanilla، Flash.

MLX-LM · APPLE SILICON

MLX-LM: سلاح مخفی Mac برای مدل‌های بزرگ

MLX فریم‌ورک یادگیری ماشین Apple برای تراشه‌های M (M2، M3، M4، M5). راز آن **Unified Memory** است: CPU و GPU یک حافظه مشترک دارند.

Mac Studio 192GB

$5,000

یک دستگاه

≈

4× H100 GPU

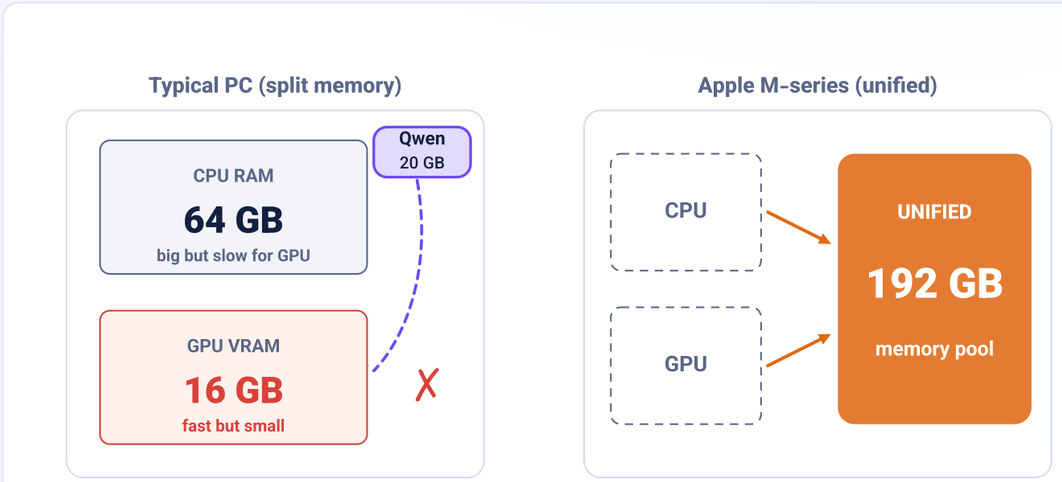
$120,000

رک + برق +

این مقایسه درباره **ظرفیت حافظه** برای جا دادن مدل‌های بزرگ است، نه سرعت. H100ها برای سرویس‌دهی هم‌زمان به کاربران زیاد بسیار سریع‌ترند؛ Mac برای یک یا چند کاربر و توسعه عالی است.

نکات

- کوانتیزه‌های آماده در **mlx-community** روی Hugging Face
- Mac Studio M3 Ultra: تا ۵۱۲ گیگ حافظه یکپارچه
- کلاستر چند Mac با **exo** برای مدل‌های خیلی بزرگ



PC معمولی: RAM بزرگ اما VRAM کوچک · Mac: یک استخر حافظه مشترک

```
pip install mlx-lm
mlx_lm.generate --model mlx-community/Qwen3.6-35B-A3B-4bit \
  --prompt "Summarize this policy in Persian"
mlx_lm.server --model ...    # OpenAI-compatible
```

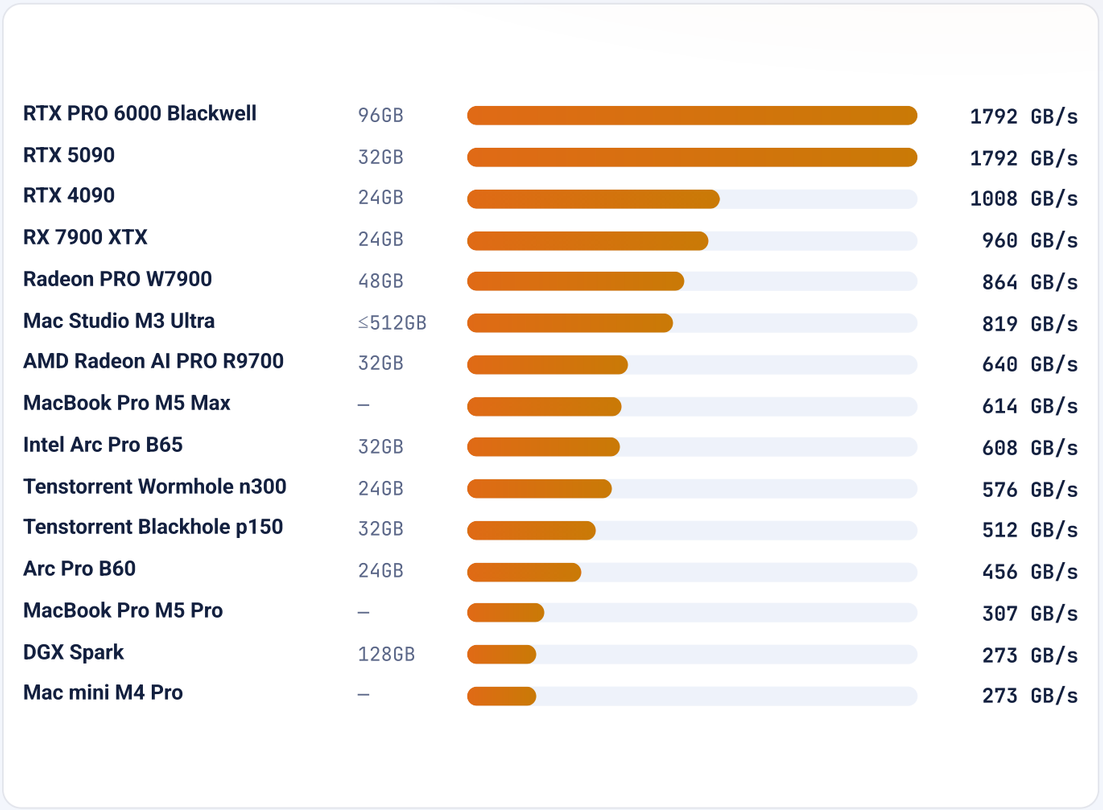
HARDWARE · BANDWIDTH

سخت‌افزار: پهنای باند حافظه، پادشاه سرعت

در تولید متن، برای هر Token باید کل وزن‌های فعال از حافظه خوانده شود. پس: *پهنای باند ÷ حجم مدل ≈ سرعت (tok/s)*. ظرفیت حافظه تعیین می‌کند **چه مدلی جا می‌شود**؛ پهنای باند تعیین می‌کند **چقدر سریع**.

دستگاه	پهنای باند	70B Q4 (35GB)	30B Q4 (15GB)	2.5GB) MoE 120B فعال)
RTX PRO 6000 / RTX 5090	1792 GB/s	~51 tok/s	~119 tok/s	~717 tok/s
RTX 4090	1008 GB/s	~29	~67	~403
RX 7900 XTX	960 GB/s	~27	~64	~384
Mac Studio M3 Ultra	819 GB/s	~23	~55	~328
DGX Spark / Mac mini M4 Pro	273 GB/s	~8	~18	~109
Strix Halo (Max+ 395)	~215 GB/s	~6	~14	~86

سقف نظری (پهنای باند ÷ حجم)؛ واقعی معمولاً ۵۰-۸۰٪. **MoE روی حافظه کند، سریع است.** RTX 5090 با ۳۲ گیگ، 70B در Q4 را کامل جا نمی‌دهد. Prefill محاسبه‌محور است، Decode حافظه‌محور.



HARDWARE · RYZEN AI MAX

Ryzen AI Max: حافظه یکپارچه ۱۲۸ گیگ روی میز

نسل جدید پردازنده‌های AMD با حافظه یکپارچه، مدل‌های بزرگ را با چند هزار دلار روی میز کار می‌آورند. پلتفرم غالب Mini PCها: **Strix Halo** (Ryzen AI Max+ 395).

دسته	نمونه تراشه	حافظه	پهنای باند	بهترین برای	70B Dense	MoE (30B-A3B)	قیمت
Ryzen معمولی	Ryzen 7 7745HS، Ryzen 9 8945HS	DDR5 دوکاناله	~50–90 GB/s	مدل 7B–13B، نمونه‌سازی RAG	قابل استفاده نیست	–	$600–$900
Ryzen AI Max 385	۸ هسته، Strix Halo ورودی	تا 32GB LPDDR5X	~256 GB/s نظری	AI سبک + کار روزمره	حافظه کافی نیست	روی MoE کوچک	~$1,099
Ryzen AI Max+ 392/388	۱۲/۸ هسته (به‌روزرسانی CES 2026)	64–96GB LPDDR5X	~256 GB/s نظری	کار LLM میان‌رده	~4–8 tok/s	~30–45 tok/s	~$1,300–$1,700
Ryzen AI Max+ 395 (128GB)	۱۶ هسته، ۳۲ رشته، ۴۰ CU iGPU	تا 128GB (~96GB برای GPU)	~215 GB/s اندازه‌گیری	70Bهای MoE، Pipeline، Agentic، Fine-tune	~4–14 tok/s	~30–55 tok/s	$1,999–$2,800

خانواده‌های Ryzen

Ryzen 3/5/7/9 (عمومی) · **Ryzen AI** (با NPU) · Ryzen Pro (سازمانی) · Ryzen Z (کنسول دستی) · Threadripper (ایستگاه کاری). برای LLM، **حافظه یکپارچه و پهنای باند** مهم است، نه تعداد هسته.

نقطه قوت

حافظه زیاد با قیمت کم ← مدل‌های MoE بزرگ با سرعت خوب؛ مصرف برق پایین؛ کوچک و بی‌صدا. عالی برای Agent شخصی یا تیم کوچک.

محدودیت

پهنای باند حدود یک‌هشتم RTX 5090 ← مدل‌های Dense بزرگ کند؛ اکوسیستم ROCm/Vulkan هنوز از CUDA عقب‌تر؛ برای کاربران هم‌زمان زیاد مناسب نیست.

HARDWARE · MINI PCs & CLUSTERS

Mini PCهای هوش مصنوعی و کلاستر کردن دستگاه‌ها

تقریباً همه سازندگان Mini PC یک مدل با Ryzen AI Max+ 395 دارند؛ NVIDIA و Apple هم گزینه‌های خودشان را دارند.

برند	مدل	پلتفرم
GMKtec	EVO-X2	Ryzen AI Max+ 395، تا 128GB
Minisforum	AI X1 Pro-470، MS-S1 Max	Ryzen AI 9 HX 470 / Max+ 395
Beelink	SER9، SER10 MAX، GTR9 Pro	Ryzen AI Max+ 395
Framework	Desktop	Ryzen AI Max 385/395، ماژولار
HP	Z2 Mini G1a	Max+ PRO 395، گارانتی سازمانی
BOSGAME	M5	Ryzen AI Max+ 395
Geekom	A9 Max	Ryzen AI 9 HX 370
AOOSTAR	Strix Halo خط	Ryzen AI Max+ 395
NVIDIA	DGX Spark	GB10 (ARM)، CUDA بومی، 128GB
Apple	Mac mini / Mac Studio	M4 Pro / M4 Max، حافظه یکپارچه

کلاستر: وقتی یک دستگاه کافی نیست

چند Mini PC یا Mac را با شبکه سریع (Thunderbolt+، 10GbE، SFP) به هم وصل کنید و مدل را بین آن‌ها تقسیم کنید؛ مثلاً با **exo** روی چند Mac mini یا RPC در llama.cpp. ظرفیت حافظه جمع می‌شود، اما شبکه گلوگاه سرعت است.

راهنمای خرید سریع

- **بودجه کم، یادگیری**: لپ‌تاپ فعلی + مدل 4B–8B
- **Agent شخصی**: Mac mini M4 Pro یا Mini PC با ۶۴ گیگ
- **تیم کوچک، MoE بزرگ**: Ryzen AI Max+ 395 یا Mac Studio
- **CUDA و Fine-tune**: RTX 4090/5090 یا DGX Spark
- **Production**: سرور GPU یا اجاره ابری

زیرساخت: انواع تراشه AI و نقشه GPUهای NVIDIA

GPU تنها گزینه نیست؛ اما برای اجرای مدل‌های باز، NVIDIA با اکوسیستم **CUDA** هنوز مسیر پیش‌فرض است. نام‌گذاری NVIDIA را بشناسید تا خرید یا اجاره درستی داشته باشید.

GPU

Graphics Processing Unit

هزاران هسته موازی؛ همه‌کاره، اکوسیستم CUDA

NVIDIA · AMD

ASIC

Application-Specific Integrated Circuit

تراشه سفارشی فقط برای AI؛ بسیار کارآمد

Google TPU · AWS Trainium/Inferentia · Meta MTIA · Microsoft Maia · OpenAI · Broadcom

FPGA

Field-Programmable Gate Array

سخت‌افزار قابل برنامه‌ریزی مجدد؛ Latency پایین

Intel · AMD (Xilinx)

Edge AI / NPU

Neural Processing Unit

کم‌مصرف، داخل لپ‌تاپ، گوشی، دستگاه صنعتی

Qualcomm · Apple · Intel · NVIDIA Jetson · MediaTek · NXP

Consumer · GeForce

بازی، تولید محتوا و LLM محلی شخصی

ورودی	GT 1030, GTX 1650
میان‌رده	GTX 1660, RTX 4060, 5060
رده بالا	RTX 4070/4080 Super, 5070 Ti
پرچم‌دار	RTX 4090, RTX 5090
بازنشسته	Titan

Workstation · RTX PRO

جانشین Quadro / RTX-A؛ CAD و توسعه AI روی میز

PRO 2000/4000	16–24GB GDDR7
PRO 4500	32GB؛ بهترین VRAM به ازای دلار
PRO 5000	72GB؛ کارهای حافظه‌محور
PRO 6000	96GB؛ قوی‌ترین GPU رومیزی

Data Center

برند «Tesla» کنار گذاشته شد؛ فقط نام مدل

Ampere	A100
Hopper	H100, H200
Blackwell	B200, B300 (288GB HBM3e)
Next	Vera Rubin (H2 2026)

Superchip / سیستم کامل

CPU + GPU آرم Grace در یک گره

DGX B200/B300	سرور چند-GPU
GB200/GB300	Grace + Blackwell؛ کارخانه‌های AI
Vera Rubin	نسل بعد، فقط Hyperscaler
DGX Spark	ابرکامپیوتر رومیزی، 128GB

SERVING · FINE-TUNING · SECURITY

استقرار، Fine-tune و امنیت مدل‌های باز

مدل باز یعنی **همه مسئولیت‌ها با شماست**: سرور، به‌روزرسانی، کیفیت و امنیت. سه موضوعی که بعد از «اجرا شد!» باید فکرش را بکنید:

استقرار · Serving

- **لپ‌تاپ**: Ollama / LM Studio؛ توسعه و دمو
- **سرور سازمان**: vLLM در Docker پشت API Gateway با احراز هویت
- **GPU اجاره‌ای**: RunPod، Vast.ai، Lambda، Modal؛ ساعتی
- **Inference API مدل باز**: Together، Fireworks، Groq، OpenRouter
- پایش: tok/s، صف، Latency (P95)، VRAM

نقطه سربه‌سر: اگر بار کم و نامنظم است، API ارزان‌تر است. اگر بار زیاد و پیوسته است یا داده حساس است، سرور خودتان.

Fine-tune · سفارشی‌سازی

- **LoRA**: فقط ماتریس‌های کوچک اضافه آموزش می‌بینند (کمتر از ۱٪ پارامترها)
- **QLoRA**: LoRA روی مدل ۴ بیتی؛ 8B روی یک GPU مصرفی
- ابزارها: MLX، Hugging Face **TRL**، **Axolotl**، **Unsloth**
- اول RAG و Prompt؛ Fine-tune برای سبک، قالب و وظیفه تکراری
- خروجی را به GGUF تبدیل و با Ollama اجرا کنید

امنیت · Supply Chain

- **safetensors** به‌جای pickle (.bin/.pt): فایل pickle می‌تواند کد اجرا کند
- trust_remote_code=True فقط برای منبع معتبر
- فقط از سازنده رسمی یا Quantizerهای شناخته‌شده
- Hash فایل و نسخه را ثبت کنید
- سرور Ollama/vLLM را بدون احراز هویت روی اینترنت باز نکنید
- **محلی ≠ امن**: Prompt Injection همچنان وجود دارد

RLHF · REINFORCEMENT LEARNING FOR LLMs

RLHF: چطور مدل «مفید و مؤدب» می‌شود؟

RLHF یعنی یادگیری تقویتی با بازخورد انسان: انسان‌ها پاسخ‌ها را مقایسه می‌کنند، یک **مدل پاداش** از آن یاد می‌گیرد و LLM برای پاداش بیشتر بهینه می‌شود (مبانی RL: جلسهٔ ۲).

۱ · SFT — آموزش با نمونه پاسخ‌های خوب

۲ · Reward Model — یادگیری ترجیح انسان از مقایسه

۳ · RL — PPO + جریمهٔ KL تا مدل از مسیر خارج نشود

PPO	RLHF کلاسیک؛ Reward Model + Critic + جریمهٔ KL. قدرتمند، اما سنگین و حساس به تنظیم
DPO	جفت ترجیح ← یک Loss؛ ساده‌ترین؛ برای هم‌سوسازی سبک و ایمنی
GRPO	مقایسهٔ گروهی پاسخ‌ها، بدون Critic؛ پایهٔ مدل‌های استدلالی
RLAIF	داور یک مدل است، نه انسان؛ Constitutional AI و داوری مدل‌محور
RLVR	پاداش از بررسی خودکار: تست کد، پاسخ ریاضی، قالب خروجی
دیگر	KTO، ORPO، SimPO، RLOO، DAPO؛ ساده‌سازی‌ها و بهبودهای همین ایده‌ها

ابزارها: Unsloth، verl، OpenRLHF، TRL (Hugging Face). روی مدل باز با LoRA هم شدنی است؛ هزینه اصلی، داده ترجیح خوب است.

۲۰۱۷ — **Deep RL from Human Preferences**
Christiano و همکاران (OpenAI و DeepMind): اولین بار انسان به‌جای تابع پاداش، دو رفتار را مقایسه می‌کند و عامل از آن یاد می‌گیرد (بازی Atari و ربات).

۲۰۲۰ — **Learning to summarize from human feedback**
Stiennon و همکاران (OpenAI): اولین کاربرد روی زبان: خلاصه‌سازی با Reward Model از مقایسه‌های انسانی.

۲۰۲۲ — **InstructGPT: Training language models to follow instructions…**
Ouyang و همکاران (OpenAI): PPO ← Reward Model ← SFT. مدل ۱.۳B از GPT-3 با ۱۷۵B بیشتر ترجیح داده شد؛ پایهٔ ChatGPT (نوامبر ۲۰۲۲).

۲۰۲۲ — **Constitutional AI: Harmlessness from AI Feedback**
Bai و همکاران (Anthropic): بازخورد از خود مدل بر پایهٔ «قانون اساسی» نوشته‌شده (RLAIF)؛ وابستگی کمتر به برچسب انسانی.

۲۰۲۳ — **Direct Preference Optimization (DPO)**
Rafailov و همکاران (Stanford): بدون Reward Model و بدون حلقهٔ RL: فقط جفت «برگزیده / ردشده» و یک Loss ساده.

۲۰۲۴ — **DeepSeekMath: GRPO**
Shao و همکاران (DeepSeek): چند پاسخ برای هر پرسش و مقایسهٔ گروهی؛ بدون Critic؛ حافظهٔ کمتر از PPO.

۲۰۲۵ — **DeepSeek-R1**
DeepSeek-AI: RL با پاداش قابل راستی‌آزمایی (RLVR): استدلال بلند از دل RL روی ریاضی و کد ظاهر شد.

HARNESS · AGENT = MODEL + HARNESS

Harness: هر چیزی که دور مدل می‌پیچد

مدل فقط متن تولید می‌کند. **Harness** کدی است که آن را به یک Agent تبدیل می‌کند: حلقه، ابزار، حافظه، مجوز و ثبت. یک مدل در دو Harness مختلف می‌تواند دو نتیجهٔ کاملاً متفاوت بدهد.

Agent = Model + Harness

- **حلقه و شرط توقف**
  Agent Loop، سقف قدم، Timeout، تشخیص حلقهٔ تکراری
- **ابزارها**
  تعریف، اعتبارسنجی، اجرا و MCP
- **Context و حافظه**
  چه چیزی کِی وارد پنجره شود؛ فشرده‌سازی؛ فایل‌های AGENTS.md و Skills
- **مجوز و Sandbox**
  چه کاری آزاد است، چه کاری تأیید می‌خواهد، کجا اجرا می‌شود
- **Hook و Guardrail**
  بررسی پیش و پس از هر ابزار، توقف، ثبت
- **Subagent و Trace**
  تقسیم کار، ثبت هر قدم، هزینه و تأخیر

Agent Harness

برنامه‌ای که مدل را در حلقه اجرا می‌کند و به فایل، ترمینال و وب وصل می‌کند:

Gemini CLI · Codex CLI · Claude Code · Goose · Cline · Aider · OpenCode · OpenHands · Hermes Agent

Eval Harness

معنای دوم: چارچوبی که یک مدل را روی بنچمارک‌ها یکسان اجرا و نمره می‌دهد:

SWE-bench · Inspect · lm-evaluation-harness

چرا مهم است؟

- نتیجهٔ بنچمارک Agent، نتیجهٔ **مدل + Harness** است، نه فقط مدل
- مدل باز بدون قالب Tool Call و Parser درست، «ابزار صدا نمی‌زند»
- Context، مجوز و Sandbox همان‌جاست که امنیت واقعی ساخته می‌شود
- تغییر Harness گاهی بیشتر از تعویض مدل کیفیت را بالا می‌برد

Agentic **Harness Engineering** مهارت تازه‌ای است: ابزار، Context و Eval را طوری طراحی کنید که یک مدل متوسط هم قابل‌اتکا شود.

PROJECTS & TOOLS WORTH KNOWING

پروژه‌ها و ابزارهایی که باید بشناسید

اکوسیستم هر هفته عوض می‌شود. این‌ها را بشناسید تا وقتی مسئله‌ای دیدید بدانید کجا دنبال راه‌حل بگردید. هر نام به سایت یا مخزن رسمی لینک است.

JEV

TypeSafe · API

مدل «فقط-قضاوت»: متن و پرسش می‌دهید (بله/خیر، گزینه، نمرهٔ ۰ تا ۱۰) و جواب را با احتمال می‌گیرید. تولید متن نمی‌کند؛ برای مسیریابی و دسته‌بندی ارزان و سریع است. هنوز Early Access.

مشابه‌های باز: Kev، Nimble، Laya

Hermes

Nous Research

خانوادهٔ مدل‌های باز با Tool Calling و دستورپذیری قوی، و «Agent :Hermes Agent» خودبهبوددهنده با مهارت و حافظه و درگاه Telegram (MIT).

مدل‌ها · Hermes Agent

Obsidian

یادداشت محلی

یادداشت‌برداری Markdown روی فایل‌های خودتان. با پلاگین به Ollama وصل می‌شود: جستجو و پرسش و پاسخ روی یادداشت‌ها، «مغز دوم» خصوصی.

Devin

Cognition

«مهندس نرم‌افزار AI» خودمختار: در محیط خودش کد می‌نویسد، تست می‌کند و PR می‌فرستد. نمونهٔ Agent سطح L4 با Sandbox.

جایگزین باز: OpenHands

Whisper

OpenAI · ASR

تبدیل گفتار به متن، متن‌باز و چندزبانه (از جمله فارسی). نسخه‌های سریع برای اجرای محلی:

whisper.cpp · faster-whisper

Open WebUI

رابط گفتگو

رابطی شبیه ChatGPT روی Ollama و هر API سازگار با OpenAI؛ کاربران، RAG و ابزار. سریع‌ترین راه دادن LLM محلی به یک تیم.

AnythingLLM

RAG محلی

اپ همه‌کاره: سند را بکشید و رها کنید، با مدل محلی گفتگو و Agent بسازید؛ Vector DB داخلی.

Docling

IBM · اسناد

تبدیل PDF و Word و جدول به Markdown و JSON ساختاریافته؛ ورودی عالی برای RAG و برای اسناد مالی پروژهٔ امروز.

smolagents

Hugging Face

کتابخانهٔ مینیمال Agent که به‌جای JSON، کد پایتون می‌نویسد؛ با مدل محلی و Ollama کار می‌کند.

LangGraph

ارکستراسیون

گراف State، Checkpoint و Routing؛ هستهٔ پروژهٔ امروز.

مستندات: docs

Streamlit

UI سریع

داشبورد و رابط تعاملی فقط با پایتون؛ مناسب دمو و ابزار داخلی.

Unsloth

Fine-tune

Fine-tune و RL (LoRA، QLoRA، GRPO) سریع‌تر و با حافظهٔ کمتر روی GPU مصرفی.

مدل محلی در پروژهٔ مالی: کدام کار را خانگی کنیم؟

سند مالی حساس است: **مدل محلی** سند خام را می‌خواند و فقط JSON به مدل‌های API می‌رسد؛ فقط base_url عوض می‌شود.

```python
import json, os
from openai import OpenAI

# the same client, pointed at Ollama (or LM Studio / vLLM)
local = OpenAI(base_url=os.environ["OLLAMA_BASE_URL"] + "/v1",
               api_key="ollama")              # ignored locally

resp = local.chat.completions.create(
    model=os.environ["OLLAMA_MODEL"],         # small, CPU-friendly
    messages=[{"role": "system", "content": EXTRACT_PROMPT},
              {"role": "user", "content": document_text}],
    response_format={"type": "json_object"},
    temperature=0)

fields = json.loads(resp.choices[0].message.content)
validate(fields)   # Pydantic; on failure: show the error, never fake it
```

Agentic **معماری ترکیبی**: هر چه داده خام دارد روی مدل محلی؛ API فقط خروجی ساختاریافته (با Mask شناسه‌ها). اگر مدل محلی نبود، خطا نشان بدهید، نه جایگزینی بی‌صدا.

چک‌لیست مدل محلی برای این پروژه

- کوچک و ۴ **CPU-friendly**: تا ۸B با Q4
- **JSON پایدار** و پیروی از Schema
- Context کافی برای یک سند (8K تا 32K)
- فارسی، انگلیسی و ارقام؛ حتماً تست کنید
- مجوز Apache/MIT؛ Eval روی نمونه‌های شناخته‌شده

تمرین تا جلسه بعد

- دموی مالی را اجرا و دو مدل محلی (کوچک‌تر و بزرگ‌تر) را روی سه سند از نظر دقت و تأخیر مقایسه کنید
- با دو API دیگر تکرار کنید و جدول داشبورد را بررسی کنید
- یک سند «Other» اضافه کنید و Routing را ببینید

◢ OPENCODE + OLLAMA · LOCAL CODING AGENT

نصب OpenCode با مدل محلی: Windows، macOS و Linux

Ollama + OpenCode + یک مدل کدنویسی محلی؛ بعد از دانلود اولیه بدون اینترنت. مراحل ۱ و ۲ در تب سیستم‌عامل، بقیه مشترک.

Windows | macOS | Linux

۱ · نصب OpenCode

اپ دسکتاپ از opencode.ai/download، یا برای CLI (روش پیشنهادی مستندات) داخل WSL:

```
wsl --install
# inside WSL:
curl -fsSL https://opencode.ai/install | bash
```

۲ · نصب Ollama

در PowerShell (ollama.com/download):

```
irm https://ollama.com/install.ps1 | iex
```

فایل تنظیمات (در صورت نیاز)

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.config\opencode"
notepad "$env:USERPROFILE\.config\opencode\opencode.json"
```

آزمایش API محلی

```
Invoke-RestMethod http://localhost:11434/api/tags
```

۳ · دانلود مدل (پیش از قطع اینترنت)

```
ollama pull qwen3.5:9b
ollama run qwen3.5:9b   # test; /bye to exit
```

۴ · اتصال خودکار (پیشنهادی، هر سه سیستم)

```
ollama launch opencode
ollama launch opencode --config   # configure only
```

۵ · اگر مدل‌ها دیده نشد: opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {"ollama": {"npm": "@ai-sdk/openai-compatible", "name": "Ollama Local",
    "options": {"baseURL": "http://localhost:11434/v1"},
    "models": {"qwen3.5:9b": {"name": "Qwen3.5 9B"}, "gpt-oss:20b": {"name": "GPT-OSS 20B"}}}}
}
```

نام مدل‌ها مثل ollama list؛ بعد /models ← Ollama Local.

✅ بدون اینترنت آزمایش کنید و ارائه‌دهندهٔ محلی را انتخاب کنید. برای پروژه‌های بزرگ، Context حداقل 64K (RAM بیشتر).

CHECKLIST 1 · MODELING A REAL-WORLD PROBLEM

چک‌لیست ۱: یک مسئلهٔ واقعی را چطور مدل کنیم؟

از جلسهٔ ۰ تا ۵ یک الگو تکرار شد: مسئله و ارزش، داده، ساده‌ترین مدل، اندازه‌گیری، و در پایان انسان و عملیات. برچسب‌های ریز، جلسهٔ مربوط را نشان می‌دهند.

۱ · مسئله و ارزش

- ☐ چه تصمیمی بهتر می‌شود؟ معیار موفقیت (KPI) و هزینهٔ خطا چیست؟
- ☐ راه‌حل بدون AI چقدر خوب است؟ این Baseline شماست
- ☐ قیدها: بودجه، تأخیر، حریم خصوصی، قانون، تحریم ج۰
- ☐ AI را فقط جایی بگذارید که ارزش اضافه دارد

۲ · نوع مسئله

- ☐ عدد ← رگرسیون · دسته ← طبقه‌بندی · بی‌برچسب ← خوشه‌بندی ج۲
- ☐ داده جدولی ← اول XGBoost؛ تصویر، صوت، گراف: خانوادهٔ مدل خودشان ج۲
- ☐ تولید و استخراج ← LLM ج۳؛ چندقدمی ← Workflow یا Agent ج۴

۳ · داده

- ☐ منبع، مجوز، حجم و کیفیت برچسب را پیش از مدل بسنجید ج۱
- ☐ EDA: Missing، Outlier، تکراری، چولگی ج۱
- ☐ Train / Validation / Test جدا؛ مراقب Data Leakage ج۱
- ☐ PII، رضایت و Data Dictionary از همان اول

۴ · مدل و ارزیابی

- ☐ از ساده‌ترین مدل شروع؛ هر پیچیدگی را Eval توجیه کند
- ☐ معیار متناسب: F1/PR برای نامتوازن، RMSE/MAE برای عدد ج۲
- ☐ Cross-Validation و بررسی Overfitting ج۲
- ☐ LLM: Golden Set، Eval مسیر و نتیجه ج۴

۵ · استقرار و پایش

- ☐ Spec اول؛ Git و Docker از قدم اول ج۳
- ☐ نسخه‌بندی داده، مدل و Prompt؛ Rollback
- ☐ Shadow ← Canary ← Production ج۴
- ☐ پایش کیفیت، هزینه، Drift؛ مالک و Kill Switch

۶ · انسان و مسئولیت

- ☐ جایی که خطا گران یا برگشت‌ناپذیر است، انسان تأیید می‌کند ج۴
- ☐ توضیح‌پذیری: شواهد، ارجاع، SHAP ج۲
- ☐ سوگیری را برای هر گروه جدا بسنجید
- ☐ اگر Eval بهبود نشان نداد، «نساختن» هم نتیجهٔ خوبی است

چک‌لیست ۲: کدام معماری؟ از ساده به پیچیده

نردبان معماری: از بالا شروع کنید و فقط وقتی پایین بروید که Eval نشان دهد پلهٔ قبل کافی نیست. بیشتر سیستم‌های موفق، ساده‌اند.

وقتی مسئله این است...		این معماری را امتحان کنید	جلسه
داده جدولی و برچسب‌دار	←	**XGBoost، Random Forest، رگرسیون**	ج۲
تصویر، صوت، گراف، سری زمانی	←	**CNN، ASR (Whisper)، GNN، LSTM**	ج۲
دسته‌بندی یا استخراج از متن؛ ثابت و پرحجم	←	**Fine-tune مدل کوچک (مثل ParsBERT) یا مدل کوچک محلی**	ج۲ ج۵
پاسخ از دانش خصوصی یا به‌روز	←	**RAG: قطعه‌بندی، Embedding، Vector DB، ارجاع**	ج۳
سبک یا قالب تکراری که RAG حل نکرد	←	**Fine-tune با LoRA/QLoRA (و گاهی DPO)**	ج۵
مسیر ثابت با چند قدم LLM	←	**Workflow: Chaining، Routing، Parallelization**	ج۴
تعداد قدم‌ها از پیش معلوم نیست	←	**Agent با ابزار، حافظه و سقف قدم**	ج۴
نقش‌ها و دسترسی‌های متفاوت	←	**چند-Agent (Supervisor) با Eval و Tracing**	ج۴
داده حساس + نیاز به مدل قوی	←	**معماری ترکیبی: مدل محلی برای PII، API برای بقیه، پشت Gateway**	ج۵

Agentic **قانون طلایی:** ساده‌ترین راه‌حلی که کار می‌کند. هر پله را با Golden Set مقایسه کنید؛ بالا رفتن بدون اندازه‌گیری، هزینه و ریسک را بالا می‌برد.

چک‌لیست ۳: انتخاب مدل (API یا محلی، بزرگ یا کوچک)

Leaderboard راهنماست، نه حکم. تصمیم نهایی را Eval روی داده خودتان می‌گیرد.

ده معیار انتخاب

- ☐ کیفیت روی Golden Set خودتان (ج۴)
- ☐ Tool Calling و JSON پایدار در حلقه‌های طولانی
- ☐ کیفیت فارسی (حتماً تست کنید)
- ☐ Context کافی (برای اسناد، حداقل ۳۲K)
- ☐ هزینه به‌ازای وظیفه و تأخیر P95 (ج۳)
- ☐ مجوز: Apache/MIT برای کار سازمانی (ج۵)
- ☐ حریم خصوصی و محل نگهداری داده
- ☐ سخت‌افزار و حافظه، پایداری نسخه، پشتیبانی

API یا محلی؟

- ☐ **API**: کیفیت بالاتر، بدون سخت‌افزار، بار کم یا نامنظم، نمونه‌سازی
- ☐ **محلی**: داده حساس، حجم بالا و ثابت، آفلاین یا تحریم، کنترل نسخه، Fine-tune (ج۵)
- ☐ هزینهٔ پنهان محلی: GPU، عملیات، به‌روزرسانی، امنیت
- ☐ نقطهٔ سربه‌سر را با عدد حساب کنید، نه با حس
- ☐ ترکیبی: Gateway مثل LiteLLM و Fallback بین مدل‌ها

اندازه و معماری مدل

- ☐ کوچک برای مسیریابی، استخراج و دسته‌بندی؛ بزرگ یا استدلالی برای تحلیل و برنامه‌ریزی
- ☐ MoE: حافظهٔ زیاد، سرعت خوب؛ Dense: ساده و قابل‌پیش‌بینی (ج۵)
- ☐ حافظه ≈ پارامتر × بایت + ۲۰٪ + KV Cache
- ☐ سرعت ≈ پهنای باند ÷ حجم مدل
- ☐ Q4_K_M پیش‌فرض خوب؛ Q8 اگر حافظه دارید

Agentic **روش پیشنهادی:** ۳ کاندید از Leaderboard و جامعه ← Golden Set با ۳۰ تا ۱۰۰ نمونهٔ واقعی ← مقایسهٔ کیفیت، هزینه و تأخیر ← ثبت تصمیم ← تکرار با هر نسخهٔ جدید.

CHECKLIST 4 · IMPLEMENTATION CONSIDERATIONS

چک‌لیست ۴: ملاحظات پیاده‌سازی و استقرار

فاصلهٔ «دمو کار کرد» تا «هر روز، برای همه، با هزینهٔ قابل قبول کار می‌کند» را این فهرست پر می‌کند.



ساختار پروژه

- ☐ SPEC.md یا AGENTS.md اول؛ سپس پرامپت‌های کوچک، قدم‌به‌قدم (ج۳)
- ☐ بعد از هر قدم Commit؛ Docker Compose از همان ابتدا
- ☐ .env و .env.example؛ هیچ سکرتی در Git
- ☐ README با دستورهای دقیق اجرا؛ تست و Lint



ساخت برنامهٔ LLM

- ☐ خروجی ساختاریافته با Pydantic یا JSON Schema (ج۴)
- ☐ Prompt نسخه‌دار؛ Context انتخاب‌شده، نه همه چیز
- ☐ Timeout، Retry با Idempotency، Backoff؛ سقف قدم و هزینه
- ☐ Fallback بین مدل‌ها (LiteLLM)؛ Streaming برای تجربهٔ بهتر



دانش و RAG

- ☐ قطعه‌بندی با هم‌پوشانی و Metadata (سند، بخش، صفحه) (ج۳)
- ☐ Embedding و Vector DB؛ بازیابی ترکیبی در صورت نیاز
- ☐ ارجاع به منبع واقعی؛ «نمی‌دانم» وقتی شاهدی نیست
- ☐ به‌روزرسانی و حذف اسناد؛ مجوز دسترسی به اسناد



Production و عملیات

- ☐ Health check، Log ساختاریافته، Tracing (OpenTelemetry، Langfuse)
- ☐ داشبورد توکن، تأخیر و کیفیت؛ بودجه و هشدار (ج۴)
- ☐ پشتیبان‌گیری و Restore آزموده؛ Rollout تدریجی و Feature Flag
- ☐ Runbook، Kill Switch حادثه، مالک مشخص برای هر Agent

CHECKLIST 5 · DEFENSE & SAFETY

چک‌لیست ۵: دفاع، ایمنی و امنیت

فرض کنید تزریق موفق می‌شود و سیستم را طوری بسازید که آسیب محدود بماند: آنچه مدل «نباید» بکند، باید در **کد ابزار** غیرممکن باشد.

تهدیدهای اصلی [ج۴]

- ☐ Prompt Injection مستقیم و غیرمستقیم (ایمیل، سند، وب، خروجی ابزار)
- ☐ سه‌گانهٔ مرگبار: داده خصوصی + محتوای ناامن + ارتباط بیرونی؛ حداقل یک ضلع را قطع کنید
- ☐ OWASP Agentic Top 10: Goal Hijack، Tool Misuse، Memory Poisoning
- ☐ Hallucination و اعتماد بیش‌ازحد انسان (Automation Bias) ج۵
- ☐ نشت سکرت و PII از طریق لاگ یا خروجی

دفاع چندلایه [ج۴]

- ☐ **ورودی:** اعتبارسنجی، حذف متن پنهان، Spotlighting
- ☐ **مدل:** دستور صریح «محتوا دستور نیست»، Temperature پایین
- ☐ **ابزار:** حداقل دسترسی، Allowlist، پارامتر محدود، کلید جدا
- ☐ **اجرا:** Sandbox، Timeout، سقف قدم و هزینه، Rate Limit
- ☐ **خروجی:** Schema، فیلتر PII، بررسی سیاست
- ☐ **انسان:** تأیید اقدام پرریسک، Kill Switch، Audit

مدل محلی و زنجیرهٔ تأمین [ج۵]

- ☐ فرمت safetensors به‌جای pickle (.bin/.pt)
- ☐ trust_remote_code=True فقط برای منبع معتبر
- ☐ فقط سازندهٔ رسمی یا Quantizer شناخته‌شده؛ Hash و نسخه را ثبت کنید
- ☐ Ollama و vLLM را بدون احراز هویت روی اینترنت باز نکنید
- ☐ «محلی» یعنی «امن» نیست؛ MCP Server شخص ثالث = کد شخص ثالث
- ☐ Red Teaming دوره‌ای با ۱۰ حملهٔ ثابت

CHECKLIST 6 · PRIVACY, REGULATION & ETHICS

چک‌لیست ۶: حریم خصوصی، قانون و اخلاق

فنی بودن کافی نیست. قبل از انتشار، این‌ها را با تیم حقوقی یا مسئول انطباق مرور کنید؛ این فهرست جایگزین مشاورهٔ حقوقی نیست.



حریم خصوصی و داده

- ☐ حداقل داده لازم؛ PII را Mask کنید پیش از ارسال به API (ج۴)
- ☐ رضایت، هدف مشخص، دورهٔ نگهداری و حذف به درخواست کاربر
- ☐ محل نگهداری داده (Data Residency)؛ قرارداد پردازش با ارائه‌دهنده
- ☐ داده حساس (HR، مالی، سلامت) ترجیحاً روی مدل محلی (ج۵)
- ☐ GDPR: حق دسترسی، اصلاح و فراموش‌شدن



قانون و استاندارد [ج۰]

- ☐ EU AI Act: سطح ریسک را تعیین کنید؛ استخدام و ارزیابی کارکنان «پرریسک» است
- ☐ NIST AI RMF و ISO/IEC 42001: چارچوب مدیریت ریسک
- ☐ شفافیت: اعلام کنید کاربر با AI طرف است؛ مستندسازی و نظارت انسانی
- ☐ اسناد و قوانین ایران را در محدودهٔ فعالیت خودتان بررسی کنید (جلسهٔ ۰)
- ☐ تحریم و محدودیت دسترسی به API: برنامهٔ جایگزین (مدل محلی، Gateway) داشته باشید



مجوز و مالکیت

- ☐ مجوز هر نسخهٔ مدل را جدا بخوانید: Research، Community، Apache/MIT (ج۵)
- ☐ Fine-tune مجوز مدل پایه را به ارث می‌برد
- ☐ مجوز داده و دیتاست (CC BY، غیرتجاری…)
- ☐ مالکیت خروجی و ذکر منبع؛ ارجاع به سند در RAG



اخلاق و انصاف

- ☐ Bias را برای هر گروه جدا اندازه بگیرید؛ از ویژگی حساس استفاده نکنید (ج۴)
- ☐ امکان اعتراض و بازبینی انسانی برای تصمیم دربارهٔ آدم‌ها
- ☐ هیچ تصمیم خودکار پرریسک بدون انسان
- ☐ محدودیت‌های سیستم را صادقانه بگویید؛ ادعای دقت بیش از Eval نکنید

CHECKLIST 7 · EVALUATION & QUALITY

چک‌لیست ۷: ارزیابی و کیفیت مداوم

بدون اندازه‌گیری، هر تغییر یک قمار است؛ هم قبل از انتشار و هم بعد از آن.

ML کلاسیک و عمیق [ج۲]

- [ ] معیار با نوع مسئله و هزینهٔ خطا جور باشد؛ Accuracy تنها کافی نیست
- [ ] RMSE، MAE، R²؛ Precision، Recall، F1، ROC-AUC
- [ ] Cross-Validation و Test نهایی فقط یک بار
- [ ] Bias–Variance، Regularization، Early Stopping
- [ ] مراقب Data Leakage و Drift بعد از انتشار ج۱

LLM و Agent [ج۴]

- [ ] Golden Set ۳۰ تا ۱۰۰ نمونه واقعی؛ در CI اجرا شود
- [ ] سطوح: Unit، Trajectory، Outcome، Safety، Human Review
- [ ] LLM-as-Judge فقط با کالیبراسیون و بازبینی انسانی
- [ ] معیار Tool Calling و پایداری در حلقهٔ طولانی ج۵
- [ ] تعداد قدم، توکن و هزینه به‌ازای وظیفه

کیفیت در Production

- [ ] بازخورد کاربر (👍 / 👎)، نرخ ویرایش و نرخ ارجاع به انسان
- [ ] Latency (P50، P95)، نرخ خطا، نرخ Timeout
- [ ] هزینهٔ روزانه، بودجه و هشدار
- [ ] بازبینی نمونه‌ای هفتگی و بازنویسی Golden Set از خطاهای واقعی

دروازهٔ انتشار

- [ ] هر تغییر مدل یا Prompt باید از Golden Set و Eval امنیتی عبور کند
- [ ] Canary ← Shadow ← کامل؛ شرط بازگشت از قبل نوشته شود
- [ ] نسخهٔ مدل قفل شود؛ تغییر بی‌خبر ارائه‌دهنده را رصد کنید
- [ ] نتیجهٔ Eval و تصمیم، در مستندات ثبت شود

چک‌لیست ۸: کل دوره در یک نگاه و آنچه نباید جا بیفتد

شش جلسه، یک زنجیره: داده ← مدل ← LLM و Agent ← RAG ← مدل باز و استقرار. آخرین مرور را از اینجا شروع کنید.

ج۰ جلسه ۰ · نقشهٔ AI

- ☐ تاریخچه، Scaling، Transformer و ChatGPT
- ☐ نقشهٔ توانایی‌ها، ریسک‌ها و قوانین
- ☐ ایران، تحریم و مسیر شغلی

ج۱ جلسه ۱ · داده

- ☐ انواع داده و قالب‌ها (Parquet، JSON)
- ☐ EDA، Missing، Outlier، Encoding، Scaling
- ☐ Split و Leakage؛ داشبورد BI

ج۲ جلسه ۲ · ML و یادگیری عمیق

- ☐ مدل‌های کلاسیک، Ensemble، خوشه‌بندی
- ☐ CNN، RNN، Transformer؛ معیارها
- ☐ Fine-tune ParsBERT؛ RL و RLHF

ج۳ جلسه ۳ · API تا RAG

- ☐ LLM، محدودیت‌ها، Prompt و Context
- ☐ RAG، Embedding، Vector DB
- ☐ SPEC، OAuth، Backend، Frontend و پرامپت

ج۴ جلسه ۴ · Agent

- ☐ Workflow یا Agent؛ ابزار و MCP
- ☐ حافظه، الگوها، چند-Agent، خودمختاری
- ☐ OWASP، Eval، Tracing؛ دستیار شخصی

ج۵ جلسه ۵ · مدل باز

- ☐ مجوز، Dense، Quantization، GGUF و MoE
- ☐ MLX، vLLM، Ollama، llama.cpp؛ سخت‌افزار
- ☐ Fine-tune، RLHF، Harness؛ دموی مالی

مهارت‌های عرضی که هر جلسه به کار آمد

Prompt و Context Engineering · MCP و A2A · Tokenomics: هزینه و تأخیر · ASR و Multimodal · Docker و Git · SQL و پایگاه داده · خواندن مقاله و مستندات · مستندسازی و ارائه

Agentic **آخرین سه سؤال برای هر پروژه:** ۱) ارزش و Baseline روشن است؟ ۲) با Eval ثابت کرده‌ایم بهتر است؟ ۳) اگر اشتباه کند، آسیب محدود است و انسان می‌بیند؟ و برای Demo Day: پورتفولیو، مستندات و یک روایت روشن.